

Enginyeria del Programari de Components i Sistemes Distribuïts

PROVA D'AVALUACIÓ CONTINUADA – Pràctica 3

Presentació

A la Pràctica 2 s'ha implementat tres microserveis del cas **Photo&Film4You**. En aquesta Pràctica 3 provarem (*testing*) un d'aquests tres microserveis, i validarem que el nostre codi segueix una arquitectura hexagonal.

La Pràctica 3 cobreix els continguts del llibres "Microservices Patterns" i "Fundamentals of Software Architecture" (veure a l'apartat *Recursos* els capítols que cal estudiar).

Competències

Aquesta pràctica treballa les següents competències del Grau d'Enginyeria Informàtica:

- Proposta i avaluació de diferents alternatives tecnològiques per resoldre un problema concret.
- Aplicació de les tècniques específiques de l'Enginyeria del Programari a les diferents etapes del cicle de vida d'un projecte.

Objectius

Els objectius de la Pràctica 3 són:

- Reconèixer els factors de testing i qualitat d'una especificació/disseny.
- Avaluar formalment el testing i la qualitat d'una implementació.
- Identificar els models més importants i estàndards de qualitat de programari.

Descripció de la Pràctica 3 a realitzar

La Pràctica 3 es compon de tres exercicis. Per cadascun d'ells cal descriure **com s'ha fet l'exercici** i aportar **instruccions** per a la seva correcta execució.

Exercici 1 (4 punts)

Partint de la solució de la Pràctica 2 publicada en l'enllaç:

<https://github.com/orgs/UOC-EPCSD-SPRING-2025/repositories>

Heu de desenvolupar **proves unitàries** emprant les eines **Junit**, **Spring Boot Test**, **Mockito** i **AssertJ**.

Notes: Podeu utilitzar com a referència per ajudar-vos a desenvolupar els tests:

- <https://github.com/anirban99/hexagonal-architecture/tree/master/src/test/java/com/example/hexagonal/architecture>
- <https://github.com/eugenp/tutorials/blob/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/>
- <https://spring.io/guides/gs/testing-web/>
- <https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing.spring-boot-applications.with-mock-environment>

Consell: Per les proves unitàries no fem ús de cap infraestructura ni dependència real. No utilitzarem comunicació HTTP, servidors, bases de dades, serveis externs o altres microserveis. Tot aspecte relacionat amb dependències s'ha de simular (amb **Mockito**), excepte les classes de domini (Product, ...).

QUÈ CAL FER

Desenvolupar les següents classes i tests:

1. **DigitalItemUnitTest**. Incloure un test sobre la classe de domini *DigitalItem* que verifiqui que en crear un nou *DigitalItem*, el seu *status* sigui AVAILABLE.
2. **DigitalSessionServiceUnitTest**. Incloure tests que verifiquin:
 - Mètode Test 1: Quan cridem al mètode *findDigitalSessionByUser* amb un *userId* existent/vàlid, obtenim correctament el llistat de sessions digitals de l'usuari.

- Mètode Test 2: Quan cridem al mètode *findDigitalSessionByUser* amb un *userId* inexistent, obtenim l'excepció que indica que l'usuari no s'ha trobat.

Notes:

- Les dependències respecte el *RestTemplate* i el *DigitalSessionRepository* han de ser simulades (*mock*).

3. **DigitalItemRestControllerUnitTest.** Incloure un test que verifiqui que si cridem a l'endpoint de *findDigitalItemBySession* obtenim correctament els ítems digitals que pertanyen a la sessió amb l'identificador especificat.

Notes:

- No es pot aixecar el servidor per fer la prova. Cal simular la infraestructura web per rebre i tractar la petició REST, així com la dependència amb *DigitalItemService*.

Solució

1.1:

DigitalItemUnitTest

```
package edu.uoc.epcsd.user.domain;

import org.junit.jupiter.api.Test;
import static org.assertj.core.api.Assertions.assertThat;

public class DigitalItemUnitTest {
    @Test
    public void whenCreatingDigitalItem_thenStatusIsAvailable() {
        // Given
        Long id = 1L;
        String description = "This is a test";
        Long lat = 37_7749L;
        Long lon = -122_4194L;
        String link = "https://test.example.com/photo/AF1Q";

        // When
        DigitalItem digitalItem = DigitalItem.builder()
            .digitalSessionId(digitalSessionId)
            .description(description)
            .lat(lat)
            .lon(lon)
            .link(link)
            .build();

        // Then
        assertThat(digitalItem.getStatus()).isEqualTo(DigitalItemStatus.AVAILABLE);
    }
}
```

1.2:

En aquest exercici tenim dues opcions (qualsevol és vàlida i compleix la funció requerida en l'enunciat): utilitzar l'extensió d'**Spring** o l'extensió de **Mockito**, que no requereix Spring.

Exemple amb extensió d'Spring:

DigitalSessionServiceUnitTest

```
package edu.uoc.epcsd.user.domain.service;

import edu.uoc.epcsd.user.domain.User;
import edu.uoc.epcsd.user.domain.DigitalSession;
import edu.uoc.epcsd.user.domain.repository.DigitalSessionRepository;
import edu.uoc.epcsd.user.domain.repository.UserRepository;
import edu.uoc.epcsd.user.domain.exception.UserNotFoundException;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;

public class DigitalSessionServiceUnitTest {

    @InjectMocks
    private DigitalSessionServiceImpl digitalSessionService;

    @Mock
    private DigitalSessionRepository digitalSessionRepository;

    @Mock
    private UserRepository userRepository;

    @BeforeEach
    public void setUp() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    public void whenFindDigitalSessionByUser_withValidIdUser_thenReturnDigitalSessionList() {
        // Given
        Long user = 1L;

        DigitalSession digital1 = DigitalSession.builder().id(1L).description("digital session test 1").location("London").link("https://test.example.com/photo/lon").userId(user).build();

        DigitalSession digital2 = DigitalSession.builder().id(2L).description("digital session test 2").location("New York").link("https://test.example.com/photo/ny").userId(user).build();

        List<DigitalSession> expectedDigitalSessions = Arrays.asList(digital1, digital2);

        GetUserResponseTest getUserResponse = new GetUserResponseTest(user, "user1", "user1@user.com", "123456789");
        ResponseEntity<GetUserResponseTest> responseEntity = new ResponseEntity<>(getUserResponse, HttpStatus.OK);

        when(userRepository.findById(user)).thenReturn(Optional.of(User.builder().id(user).build()));
        when(digitalSessionRepository.findDigitalSessionByUser(user)).thenReturn(expectedDigitalSessions);

        // When
    }
}
```

```

        List<DigitalSession> actualDigitalSession = digitalSessionService.findDigitalSessionByUser(user);

        // Then
        assertEquals(expectedDigitalSessions, actualDigitalSession);
        verify(userRepository, times(1)).findUserById(user);
        verify(digitalSessionRepository, times(1)).findDigitalSessionByUser(user);
    }

    @Test
    public void whenFindDigitalSessionByUser_withInvalidIdUser_thenThrowUserNotFoundException() {
        // Given
        Long user = 11111L;

        when(userRepository.findUserById(user)).thenReturn(Optional.empty());

        // When / Then
        assertThrows(UserNotFoundException.class, () -> digitalSessionService.findDigitalSessionByUser(user));
        verify(userRepository, times(1)).findUserById(user);
        verify(digitalSessionRepository, times(0)).findDigitalSessionByUser(user);
    }
}

```

1.3:

Afegim la configuració per fer el testing únicament del Controller indicat en l'enunciat:

DigitalItemRestControllerUnitTest

```

package edu.uoc.epcsd.user.application.rest;

import edu.uoc.epcsd.user.domain.DigitalItem;
import edu.uoc.epcsd.user.domain.service.DigitalItemService;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.MockitoAnnotations;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.AutoConfigureDataJpa;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Arrays;
import java.util.List;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(DigitalItemRestController.class)
public class DigitalItemRestControllerUnitTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private DigitalItemService digitalItemService;

    @BeforeEach
    public void setUp() {
        MockitoAnnotations.openMocks(this);
    }
}

```

```
@Test
public void whenFindDigitalItemsBySession_withValidSessionId_thenReturnDigitalItemsList() throws Exception {
    // Given
    Long sessionId = "1L";

    DigitalItem item1 = DigitalItem.builder().id(1L).description("digital item test
1").lat(37_7749L).lon(-122_4194L).link("https://test.example.com/photo/lon").digitalSessionId(sessionId).build();
    DigitalItem item2 = DigitalItem.builder().id(2L).description("digital item test
2").lat(33_5894L).lon(-122_1449L).link("https://test.example.com/photo/ny").digitalSessionId(sessionId).build();

    List<DigitalItem> expectedDigitalItems = Arrays.asList(item1, item2);

    when(digitalItemService.findDigitalItemBySession(sessionId)).thenReturn(expectedDigitalItems);

    // When & Then
    mockMvc.perform(get("/digitalItem/digitalItemBySession")
        .param("digitalSessionId", "1")
        .contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        // DigitalItem1
        .andExpect(jsonPath("$.id").value(item1.getId()))
        .andExpect(jsonPath("$.description").value(item1.getDescription()))
        .andExpect(jsonPath("$.lat").value(item1.getLat()))
        .andExpect(jsonPath("$.lon").value(item1.getLon()))
        .andExpect(jsonPath("$.link").value(item1.getLink()))
        .andExpect(jsonPath("$.digitalSessionId").value(item1.getDigitalSessionId()))
        // DigitalItem2
        .andExpect(jsonPath("$.id").value(item2.getId()))
        .andExpect(jsonPath("$.description").value(item2.getDescription()))
        .andExpect(jsonPath("$.lat").value(item2.getLat()))
        .andExpect(jsonPath("$.lon").value(item2.getLon()))
        .andExpect(jsonPath("$.link").value(item2.getLink()))
        .andExpect(jsonPath("$.digitalSessionId").value(item2.getDigitalSessionId()))
}
```

Exercici 2 (4 punts)

Partint de la solució de la PRAC2 publicada a:

<https://github.com/UOC-EPCSD-SPRING-2025/repositories>

Heu de desenvolupar **proves d'integració**, utilitzant **JUnit**, **Spring Boot Test** i **AssertJ**.

Notes:

- Podeu usar com a referència per ajudar-vos a desenvolupar els tests: <https://github.com/eugenp/tutorials/tree/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/>

Consell: Les proves d'integració han de provar que el servei es comunica amb les seves dependències reals i la infraestructura. Per tant, no es fa ús de la llibreria **Mockito** com en les proves unitàries.

QUÈ CAL FER

Desenvolupar la següent classe amb el seu test:

1. **DigitalItemRepositoryIntegrationTest.** Provar que després d'afegir una *DigitalSession* amb alguns *DigitalItems*, podem recuperar aquests correctament mitjançant el mètode *findDigitalItemBySession*.

Notes:

- En aquest cas serà necessari una base de dades real per verificar que la integració funciona correctament.
2. **DigitalItemRestControllerIntegrationTest.** Provar que després d'afegir una *DigitalSession* amb alguns *DigitalItems*, podem recuperar aquests correctament mitjançant una petició HTTP a la ruta que exposa el *DigitalItemRestController* per obtenir els *DigitalItems* de la *DigitalSession*.

Notes:

- En aquest cas NO s'ha de fer la crida al mètode del Controlador, si no que cal fer directament la petició HTTP al servei web de l'aplicació Spring.

Solució

2.1:

Afegim la configuració per realitzar el testing únicament dels repositoris indicats en l'enunciat.

DigitalItemRepositoryIntegrationTestConfig

```
package edu.uoc.epcsd.user.infrastructure.repository.jpa;

import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.FilterType;

@Configuration
@EnableAutoConfiguration
@ComponentScan(basePackages = "edu.uoc.epcsd.user.infrastructure.repository.jpa",
excludeFilters = @ComponentScan.Filter(type = FilterType.ASSIGNABLE_TYPE))
public class DigitalItemRepositoryIntegrationTestConfig {
}
```

DigitalItemRepositoryIntegrationTest

```
package edu.uoc.epcsd.user.infrastructure.repository.jpa;

import edu.uoc.epcsd.user.domain.DigitalItem;
import edu.uoc.epcsd.user.domain.DigitalSession;
import edu.uoc.epcsd.user.domain.repository.DigitalItemRepository;
import edu.uoc.epcsd.user.domain.repository.DigitalSessionRepository;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.test.context.ContextConfiguration;

import java.util.List;

import static org.assertj.core.api.AssertionsForClassTypes.assertThat;

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@ContextConfiguration(classes = DigitalItemRepositoryIntegrationTestConfig.class)
public class DigitalItemRepositoryIntegrationTest {

    @Autowired
    private DigitalSessionRepository digitalSessionRepository;

    @Autowired
    private DigitalItemRepository digitalItemRepository;

    @Test
    void whenFindBySession_thenReturnDigitalSession() {
        // Given
    }
```



```

DigitalSession digitalSession = DigitalSession.builder().
    description("This is a test").
    location("Test location").
    link("https://test.example.com/session/AF1Q").
    userId(3L).
    build();

Long digitalSessionId = digitalSessionRepository.
    createDigitalSession(digitalSession);

DigitalItem digitalItem = DigitalItem.builder().
    description("This is a test").
    lat(37_7749L).
    lon(-122_4194L).
    link("https://test.example.com/photo/AF1Q").
    digitalSessionId(digitalSessionId).
    build();

Long digitalItemId = digitalItemRepository.
    createDigitalItem(digitalItem);

// When
List<DigitalItem> fromDb = digitalItemRepository.
    findDigitalItemBySession(digitalSessionId);

// Then
assertThat(fromDb.size()).isEqualTo(1);

digitalItem.setId(digitalItemId);
assertThat(fromDb.contains(digitalItem)).isEqualTo(true);
}
}

```

2.2:

Afegim la configuració necessària pel testing de la integració real dels serveis del controlador indicat en l'enunciat

DigitalItemRestControllerTestConfig

```

package edu.uoc.epcsd.user.application.rest;

import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.FilterType;
import org.springframework.web.client.RestTemplate;

@Configuration
@EnableAutoConfiguration
@ComponentScan(basePackages = {
    "edu.uoc.epcsd.user.application.rest",
    "edu.uoc.epcsd.user.domain.service",
    "edu.uoc.epcsd.user.infrastructure.repository",

```

```

        "edu.uoc.epcsd.user.infrastructure.repository.jpa",
    }, excludeFilters = @ComponentScan.Filter(type = FilterType.ASSIGNABLE_TYPE))
    public class DigitalItemRestControllerTestConfig {
        @Bean
        public RestTemplate restTemplate() {
            return new RestTemplate();
        }
    }
}

```

DigitalItemRestControllerIntegrationTest

```

package edu.uoc.epcsd.user.application.rest;

import edu.uoc.epcsd.user.domain.DigitalItem;
import edu.uoc.epcsd.user.domain.DigitalSession;

import edu.uoc.epcsd.user.infrastructure.repository.jpa.DigitalItemRepositoryImpl;
import edu.uoc.epcsd.user.infrastructure.repository.jpa.DigitalSessionRepositoryImpl;

import org.junit.jupiter.api.Test;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.web.server.LocalServerPort;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import java.util.Arrays;
import java.util.List;

import static org.assertj.core.api.AssertionsForClassTypes.assertThat;

@SpringBootTest(classes = DigitalItemRestControllerTestConfig.class, webEnvironment =
    SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
public class DigitalItemRestControllerIntegrationTest {
    @LocalServerPort
    private int port;

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private DigitalSessionRepositoryImpl digitalSessionRepository;

    @Autowired
    private DigitalItemRepositoryImpl digitalItemRepository;

    @Test
    public void whenFindDigitalItemsBySession_withValidSessionId_thenReturnDigitalItemsList() {
        // Given
        DigitalSession digitalSession = DigitalSession.builder().
            description("This is a test").

```

```

        location("Test location").
        link("https://test.example.com/session/AF1Q").
        userId(3L).
        build();

        Long digitalSessionId =
digitalSessionRepository.createDigitalSession(digitalSession);

        DigitalItem item1 = DigitalItem.builder()
            .description("digital item test 1")
            .lat(37_7749L)
            .lon(-122_4194L)
            .link("https://test.example.com/photo/lon")
            .digitalSessionId(digitalSessionId)
            .build();

        DigitalItem item2 = DigitalItem.builder()
            .description("digital item test 2")
            .lat(33_5894L)
            .lon(-122_1449L)
            .link("https://test.example.com/photo/ny")
            .digitalSessionId(digitalSessionId)
            .build();

        digitalItemRepository.createDigitalItem(item1);
        digitalItemRepository.createDigitalItem(item2);

        // When
        String url = "http://localhost:" + port +
"/digitalItem/digitalItemBySession?digitalSessionId=" + digitalSessionId;
        ResponseEntity<DigitalItem[]> response = restTemplate.getForEntity(url,
DigitalItem[].class);

        // Then
        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);

        List<DigitalItem> items = Arrays.asList(response.getBody());
        assertThat(items.size()).isEqualTo(2);

        assertThat(items.get(0).getDigitalSessionId()).isEqualTo(1L);
        assertThat(items.get(1).getDigitalSessionId()).isEqualTo(1L);
    }
}

```

Exercici 3 (2 punts)

Partint de la solució de la PRAC2 publicada a:

<https://github.com/orgs/UOC-EPCSD-SPRING-2025/repositories>

Volem verificar que l'arquitectura es basa correctament en una arquitectura hexagonal, i que fem servir bones regles arquitectòniques.

Lectures recomanades:

- Capítol "Measuring and Governing Architecture Characteristics" del llibre "Fundamentals of Software Architecture"
- Documentació ArchUnit:
https://www.archunit.org/userguide/html/000_Index.html#_junit_4_5_support
- Exemples JUnit5:
<https://github.com/TNG/ArchUnit-Examples/tree/main/example-junit5/src/test>

QUÈ CAL FER

Desenvolupar els següents dos tests automàtics de verificació de l'arquitectura sobre el microservei *UserService*:

- Es compleix l'arquitectura hexagonal (anomenada *onion* a **ArchUnit**)
- Les classes que es troben al *package domain.service* i estan anotades amb **@Service**, tenen el seu nom acabat en **ServiceImpl**.

Solució

UserArchitectureTest:

```
package edu.uoc.epcsd.user;

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;
import org.springframework.stereotype.Service;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.library.Architectures.onionArchitecture;

@AnalyzeClasses(packages = "edu.uoc.epcsd.user", importOptions =
{ImportOption.DoNotIncludeTests.class, ImportOption.DoNotIncludeJars.class})
public class UserArchitectureTest {

    @ArchTest
    static final ArchRule domain_service_with_spring_annotation = classes()
        .that().resideInAPackage("..domain.service..")
        .and().areAnnotatedWith(Service.class)
        .should().haveSimpleNameEndingWith("ServiceImpl");

    @ArchTest
    static final ArchRule onion_architecture_is_respected = onionArchitecture()
        .domainModels("..domain..")
        .domainServices("..domain.service..")
        .applicationServices("..application..")
        .adapter("persistence", "..infrastructure.repository..")
        .adapter("rest", "..application.rest..");
```

Recursos

Recursos Bàsics

- Llibre "Microservices Patterns" (Chris Richardson): Chapter 9. "Testing microservices: Part 1". Introduction.
- Llibre "Microservices Patterns" (Chris Richardson): Chapter 10. "Testing microservices: Part 2".
- Llibre "Fundamentals of Software Architecture" (Richards & Ford):
 - o Chapter 3: "Modularity"
 - o Chapter 6: "Measuring and Governing Architecture Characteristics"

Criteris d'avaluació

- En aquesta activitat no està permès utilitzar **eines d'intel·ligència artificial**, i cal realitzar-la de manera **estrictament individual**. En cas de detectar irregularitats es penalitzarà la prova amb una D com a nota. Això inclou la reutilització de solucions d'aquesta pràctica de semestres anteriors. Al pla docent i al [web sobre integritat acadèmica i plagi de la UOC](#) trobareu informació sobre què es considera conducta irregular en l'avaluació i les conseqüències que pot tenir.
- El pes de cada exercici està indicat en l'enunciat.
- Trobareu una descripció ampliada dels criteris d'avaluació de cada exercici en l'annex.
- Cal justificar la solució a cadascun dels exercicis. Es valorarà tant la correcció de la solució com la justificació donada

Format i data de lliurament

Cal lliurar un únic **document** PDF *CognomsNom_EPCSDPRAC3.pdf* amb la descripció de com s'ha realitzat la pràctica (argumentar decisions i instruccions de com s'han d'executar les proves desenvolupades) i lliurar el **codi dels tests desenvolupats** (en un fitxer ZIP).

El document PDF i el fitxer ZIP s'han de lliurar a l'espai "Lliurament de la PRAC3" de l'aula abans de les **23:59 hores del dia 9 de juny de 2025**. No s'acceptaran lliuraments fora de termini.

Annex

Criteris d'avaluació de la Pràctica 3

	Excel·lent (10 punts)	Notable (7 punts)	Suficient (5 punts)	Insuficient (2,5 punts)	Sense resposta / Tot malament (0 punts)
Ex. 1	4 tests implementats i justificats correctament als exercicis 1.1, 1.2 i 1.3. Ús correcte de Junit a tots els 4 tests dels exercicis 1.1, 1.2 i 1.3. Ús correcte de Mockito als 3 tests dels exercicis 1.2 i 1.3. Separar en 2 mètodes els dos tests de l'exercici 1.2. Qualitat general de la implementació <u>perfecte</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcoded en les crides als mètodes, etc.). (4 punts)	3 tests implementats i justificats correctament als exercicis 1.1, 1.2 i 1.3. Ús correcte de Junit a tots els 4 tests dels exercicis 1.1, 1.2 i 1.3. Ús correcte de Mockito en 2 dels 3 tests dels exercicis 1.2 i 1.3. Separar en 2 mètodes els dos tests de l'exercici 1.2. Qualitat general de la implementació <u>bona</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcoded en les crides als mètodes, etc.). (3 punts)	2 tests implementats i justificats correctament als exercicis 1.1, 1.2 i 1.3. Ús correcte de Junit en 2 dels 4 tests dels exercicis 1.1, 1.2 i 1.3. Ús correcte de Mockito en 1 dels 3 tests dels exercicis 1.2 i 1.3. No separar en 2 mètodes els dos tests de l'exercici 1.2. Qualitat general de la implementació <u>suficient</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcoded en les crides als mètodes, etc.). (2 punts)	1 test implementat i justificat correctament als exercicis 1.1, 1.2 i 1.3. Ús incorrecte de Junit en els 4 tests dels exercicis 1.1, 1.2 i 1.3. No ús de Mockito en els 3 tests dels exercicis 1.2 i 1.3. No separar en 2 mètodes els dos tests de l'exercici 1.2. Qualitat general de la implementació <u>insuficient</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcoded en les crides als mètodes, etc.). (1 punt)	Sense resposta, no hi ha cap justificació o tot incorrecte. (0 punts)
Ex. 2	2 tests implementats i justificats correctament en els ex 2.1 i 2.2 Qualitat general de la implementació <u>perfecte</u> (criteris:	2 tests implementats correctament, però justificats incorrectament. Qualitat general de la implementació <u>bona</u> (criteris:	1 test implementat i justificat correctament. Qualitat general de la implementació <u>suficient</u> (criteris: llegibilitat, formatat, sense warnings,	1 test implementat però justificat incorrectament. Qualitat general de la implementació <u>insuficient</u> (criteris: llegibilitat, formatat, sense warnings,	Sense resposta, no hi ha cap justificació o tot incorrecte. (0 punts)

	llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (4 punts)	llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (3 punts)	utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (2 punts)	utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (1 punt)	
Ex. 3	2 tests implementats i justificats correctament. Qualitat general de la implementació <u>perfecte</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (2 punts)	2 tests implementats i però justificats incorrectament. Qualitat general de la implementació <u>bona</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (1,5 punts)	1 test implementat i justificat correctament. Qualitat general de la implementació <u>suficient</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (1 punts)	Tests mal implementats i no justificats. Qualitat general de la implementació <u>insuficient</u> (criteris: llegibilitat, formatat, sense warnings, utilitza constants en comptes de hardcode en les crides als mètodes, etc.). (0,5 punts)	Sense resposta, no hi ha cap justificació o tot incorrecte. (0 punts)